ASSIGNMENT 1 – TASK 10: Organise Classes Through Packages and Access Modifiers

Bloom's Level: K4 – Analyse | Marks: 3

The alternative design exposes public fields and relies on every caller to validate them. This is risky because any caller could set an invalid floor, negative fee, blank registration or inconsistent occupancy. Prefer private fields and validated setters.

```
class ParkingSlot {
    private int floor;
    private int slotNumber;
    private String vehicleNo;

    public void setFloor(int floor) {
        if (floor < 0) throw new IllegalArgumentException("Invalid floor");
        this.floor = floor;
    }

    public void setSlotNumber(int slotNumber) {
        if (slotNumber < 0) throw new IllegalArgumentException("Invalid slot");
        this.slotNumber = slotNumber;
    }

    public void setVehicleNo(String vehicleNo) {
        if (vehicleNo != null && vehicleNo.trim().isEmpty()) {
            throw new IllegalArgumentException("Blank registration");
        }
        this.vehicleNo = vehicleNo;
    }
}
```

Public fields with caller validation make every caller responsible for remembering all rules; one forgotten check corrupts shared state. Private fields centralise validation. Packages can still separate model, pricing, service and app classes, but the central issue in this alternative is encapsulation.

Wrapper-class risk: Integer is an object, not a primitive. `Integer a = 1000; Integer b = 1000; a == b` compares references and may be false. Use `a.equals(b)` or `a.intValue() == b.intValue()` for numeric equality. Do not rely on the small-value Integer cache.

Evidence: fields are private, setters validate values, and equality checks use value comparison rather than `==` on wrappers.